

Frontend-Backend Workflow Analysis

Overview

Your project has a **dual-mode architecture** where the frontend can run either:

- 1. **Server Mode** (`useServer: true`) - Connects to Express backend via HTTP polling
- 2. **Local Worker Mode** (`useServer: false`) - Runs simulation in a Web Worker

The default is **Server Mode** (defined in [gameStore.ts](#)).

Initialization Flow

```
sequenceDiagram
    participant Frontend as Frontend (App.tsx)
    participant Store as Zustand Store
    participant Server as Backend (WorldServer)
    participant Sim as SimulationState

    Note over Server: On startup
    Server->>Sim: createSimulation(seed, "server_world_v1", rules)
    Server->>Sim: chunkManager.initializeWorld(sim)
    Note over Sim: Spawns initial animals & resources

    Server->>Server: Start tick loop (30 Hz)

    Note over Frontend: On mount
    Frontend->>Store: Check useServer flag
    alt Server Mode
        Frontend->>Server: GET /api/world/snapshot (every 200ms)
        Server-->>Frontend: { tick, entities, objects, stats, events }
        Frontend->>Store: updateFromSnapshot(snapshot)
    else Worker Mode
        Frontend->>Worker: postMessage(INIT_WORLD)
        Worker-->>Frontend: SNAPSHOT messages
    end
end
```

Key Files

Component	File	Purpose
Frontend Entry	App.tsx	Mode switching, polling loop
Server Client	ServerClient.ts	HTTP fetch wrapper
Backend Entry	index.ts	Express routes, Firebase init
World Server	WorldServer.ts	Simulation lifecycle

Where Is the Map Saved?

[!IMPORTANT] *The map is NOT persistently saved by default. It lives in memory.*

Storage Layer	What's Stored	Current Status
In-Memory (WorldServer.sim)	Live simulation state (entities, objects, stats)	✔ Active
Firebase Firestore	Player profiles, world metadata, chunk stats	✔ Endpoints exist
Google Cloud Storage (GCS)	Full world snapshots (tick-based)	✔ Endpoints exist

Snapshot Save/Load (GCS)

```
POST /api/world/snapshot    → Saves current state to GCS
GET  /api/world/snapshot/:tick → Loads a specific tick
GET  /api/world/snapshots    → Lists available snapshots
```

The [SnapshotService.ts](#) handles GCS operations.

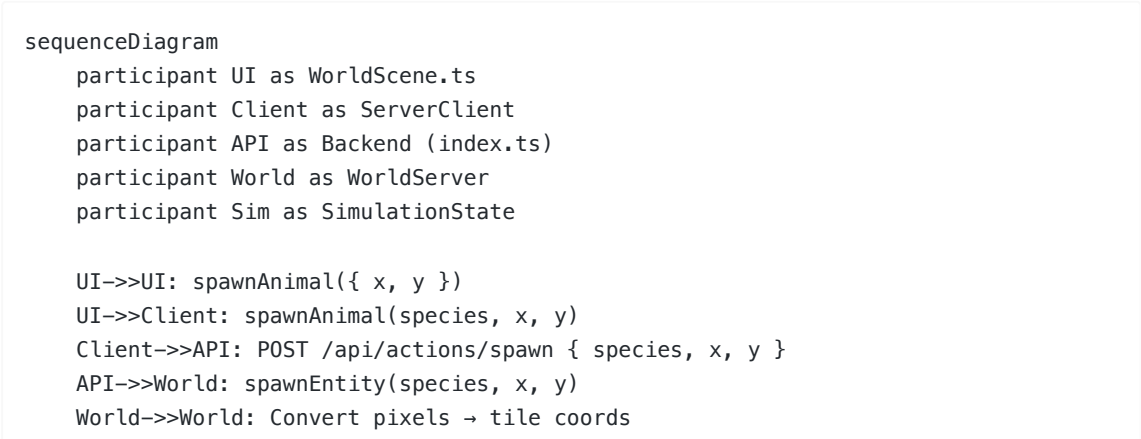
How Often Does Frontend Request Backend?

Mode	Poll Rate	Endpoint
Server Mode	200ms (5 Hz)	GET /api/world/snapshot
Worker Mode	N/A	Direct postMessage

Defined in [App.tsx:77](#):

```
pollInterval = setInterval(async () => {
  const snapshot = await ServerClient.getInstance().getSnapshot();
  // ...
}, 200); // 5Hz Polling
```

Spawn Animal Flow



```

World-->>Sim: spawnEntity(sim, species, ...)
Sim-->>Sim: entities.set(id, entity)
World-->>API: { success: true, entityId }
API-->>Client: { ok: true, entityId }
Note over UI: Next poll (200ms later)
UI-->>Client: getSnapshot()
Client-->>API: GET /api/world/snapshot
API-->>Client: { entities: [...newEntity...] }
Client-->>UI: updateFromSnapshot()

```

Key Points

1. **Spawn is instant** - Added to in-memory `sim.entities` Map immediately
2. **NOT auto-persisted** - Entity only exists in RAM until server restarts
3. **Visible on next poll** - Frontend sees new entity within 200ms
4. **Position conversion**: Pixel coords → Tile coords (divide by `V1.tileSizePx = 16`)

Backend Tick Rate

Setting	Value	Location
Tick Rate	30 Hz (33ms)	WorldServer.ts:10

```
private readonly TICK_RATE_MS = 1000 / 30; // 30 Hz
```

Available Services

Service	File	Purpose
PlayerService	PlayerService.ts	Firestore: player profiles, GP, pins, rate limits
WorldService	WorldService.ts	Firestore: world metadata
ChunkService	ChunkService.ts	Firestore: chunk stats, static objects
SnapshotService	SnapshotService.ts	GCS: full world snapshots
SSEManager	SSEManager.ts	Server-Sent Events (not used by frontend yet)

Summary

Question	Answer
Where is map saved?	In-memory only (can optionally save to GCS via API)
How often does frontend poll?	Every 200ms (5 Hz)
After spawn, how is it saved?	Added to in-memory Map, NOT auto-persisted
Backend tick rate?	30 Hz

Persistence options?

Firestore (metadata) + GCS (full snapshots)